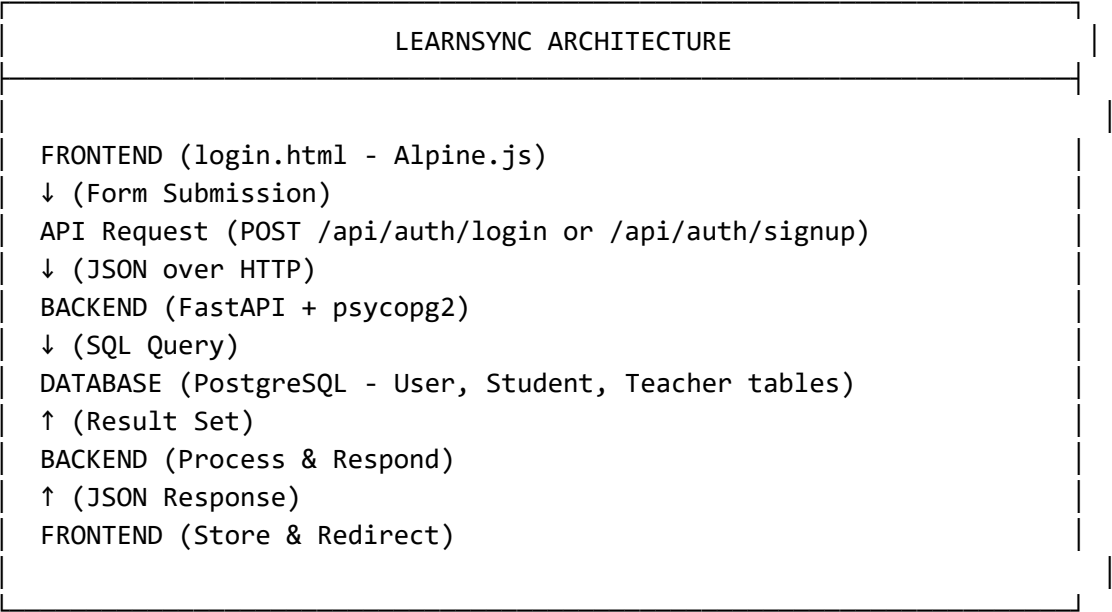


# LearnSync Authentication Flow & Architecture

## System Overview

...



...

---

## 1. LOGIN FLOW (Detailed Step-by-Step)

### Frontend (login.html - Alpine.js)

```
``javascript
// Step 1: User fills form and clicks "Log In" button
{
  role: "student",           // Selected from dropdown
  email: "juan@example.com",
  password: "SecurePass123!"
}

// Step 2: Alpine.js event handler triggers
submitLogin() → submitLoginToBackend()

// Step 3: Fetch API sends HTTP POST request
fetch('/api/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    role: "student",
    email: "juan@example.com",
    password: "SecurePass123!"
  })
})
})
```

...

### Backend (FastAPI - main.py)

```
```python
# Step 4: Request arrives at backend endpoint
@app.post("/api/auth/login", response_model=LoginResponse)
async def login(request: LoginRequest):

    # Step 5: Get database connection
    conn = get_db_connection() # Opens PostgreSQL connection
    cur = conn.cursor()

    # Step 6: Query database for user
    cur.execute(
        'SELECT user_id, email, password, role, name FROM "User"
        WHERE email = %s AND role = %s',
        ("juan@example.com", "student")
    )
    user = cur.fetchone()

    # SQL Query sent to PostgreSQL:
    # SELECT user_id, email, password, role, name
    # FROM "User"
    # WHERE email = 'juan@example.com' AND role = 'student'
```

...

### Database (PostgreSQL)

```
```sql
-- Step 7: PostgreSQL searches User table
-- Database contains:
```

| user_id | email             | password    | role    | name           |
|---------|-------------------|-------------|---------|----------------|
| 1       | juan@example.com  | a3f5d2... ✓ | student | Juan Dela Cruz |
| 2       | maria@example.com | b7c8e9...   | teacher | Maria Santos   |

```
-- Step 8: PostgreSQL returns matching record
-- Result: 1 row found with user_id=1
```

...

### Backend (Continued)

```
```python
# Step 9: Verify password
# Hash the submitted password and compare with stored hash
if verify_password("SecurePass123!", "a3f5d2..."):
    # Password matches! ✓
    pass
else:
    # Password incorrect! X
```

```

        raise HTTPException(401, "Invalid email or password")

# Step 10: Prepare response
return LoginResponse(
    success=True,
    message="Welcome back, Juan Dela Cruz!",
    user_id=1,
    role="student",
    name="Juan Dela Cruz"
)

# Step 11: Return JSON response to frontend
...

```

### ### Frontend (Response Handling)

```

```javascript
// Step 12: Response arrives at frontend
{
  "success": true,
  "message": "Welcome back, Juan Dela Cruz!",
  "user_id": 1,
  "role": "student",
  "name": "Juan Dela Cruz"
}

// Step 13: Store user data in browser localStorage
localStorage.setItem('user_id', '1');
localStorage.setItem('role', 'student');
localStorage.setItem('name', 'Juan Dela Cruz');

// Step 14: Redirect to dashboard
window.location.href = './student/dashboard.html';
...

```

---

## ## 2. SIGNUP FLOW (Detailed Step-by-Step)

### ### Frontend Form Data

```

```javascript
{
  role: "student",
  email: "newstudent@example.com",
  firstName: "Carlos",
  middleName: "Manuel",
  lastName: "Reyes",
  idNumber: "CS-24-1-567",
  password: "NewPass123!"
}
...

```

### ### Backend Processing

```
```python
# Step 1: Receive signup request
@app.post("/api/auth/signup")
async def signup(request: SignupRequest):

    # Step 2: Check if email already exists
    cur.execute('SELECT user_id FROM "User" WHERE email = %s',
                ("newstudent@example.com",))
    if cur.fetchone():
        raise HTTPException(400, "Email already registered")

    # Step 3: Hash the password
    hashed_password = hash_password("NewPass123!")
    # Result: "f7e4c2a9d..." (irreversible SHA-256 hash)

    # Step 4: Create full name
    full_name = "Carlos Manuel Reyes"

    # Step 5: Insert into User table
    cur.execute(
        '''INSERT INTO "User" (email, name, password, role, created_at)
        VALUES (%s, %s, %s, %s, %s)
        RETURNING user_id''',
        ("newstudent@example.com", "Carlos Manuel Reyes",
        "f7e4c2a9d...", "student", datetime.now())
    )
    user_id = 3 # Auto-generated primary key

    # Step 6: Create Student record linked to User
    cur.execute(
        'INSERT INTO Student (user_id) VALUES (%s)',
        (3,)
    )

    # Step 7: Commit transaction (save all changes)
    conn.commit()
```
```

### ### Database State After Signup

```
```sql
-- New User record added:
```

| user_id | email                  | password    | role    | name                | created_at          |
|---------|------------------------|-------------|---------|---------------------|---------------------|
| 3       | newstudent@example.com | f7e4c2a9d.. | student | Carlos Manuel Reyes | 2024-01-15 10:30:45 |

```
-- New Student record created:
```

| student_id | user_id | grade_level |
|------------|---------|-------------|
| 1          | 3       | NULL        |

...

---

### ## 3. Data Structure Reference

#### ### Request Models (Frontend → Backend)

##### #### LoginRequest

```
```json
{
  "role": "student|teacher",
  "email": "user@example.com",
  "password": "PlainTextPassword123!"
}
```
```

##### #### SignupRequest

```
```json
{
  "role": "student|teacher",
  "email": "user@example.com",
  "firstName": "John",
  "middleName": "Paul",
  "lastName": "Doe",
  "idNumber": "Xx-00-0-000",
  "password": "PlainTextPassword123!"
}
```
```

#### ### Response Model (Backend → Frontend)

##### #### LoginResponse

```
```json
{
  "success": true,
  "message": "Welcome back, John Doe!",
  "user_id": 1,
  "role": "student",
  "name": "John Paul Doe"
}
```
```

#### ### Database Tables

#### #### User Table

```
```sql
CREATE TABLE "User" (
  user_id SERIAL PRIMARY KEY,          -- Auto-incrementing ID
  name VARCHAR(100),                  -- Full name
  email VARCHAR(100) UNIQUE NOT NULL, -- Must be unique
  username VARCHAR(100),              -- Optional username
  password VARCHAR(255) NOT NULL,     -- SHA-256 hash
  role VARCHAR(50) NOT NULL,          -- "student" or "teacher"
  created_at TIMESTAMP                -- Registration timestamp
);
```
```

#### #### Student Table (linked to User)

```
```sql
CREATE TABLE Student (
  student_id SERIAL PRIMARY KEY,
  user_id INT,                      -- Foreign key to User
  grade_level VARCHAR(50),
  FOREIGN KEY (user_id) REFERENCES "User"(user_id)
);
```
```

#### #### Teacher Table (linked to User)

```
```sql
CREATE TABLE Teacher (
  employee_id SERIAL PRIMARY KEY,
  user_id INT,                      -- Foreign key to User
  FOREIGN KEY (user_id) REFERENCES "User"(user_id)
);
```
```

---

## ## 4. Code Snippets Explained

### ### Frontend: Making the API Request

```
```javascript
async submitLoginToBackend() {
  // 1. Create fetch request
  const response = await fetch('/api/auth/login', {
    method: 'POST',          // HTTP method
    headers: {
      'Content-Type': 'application/json' // Tell backend it's JSON
    },
    body: JSON.stringify({    // Convert JS object to JSON
      role: this.login.role,
      email: this.login.email,
      password: this.login.password
    })
  })
}
```

```

});

// 2. Parse response
const data = await response.json();

// 3. Check if successful
if (!response.ok) {
    alert(`Login failed: ${data.detail}`);
    return;
}

// 4. Store user data locally
localStorage.setItem('user_id', data.user_id);    // Browser storage
localStorage.setItem('role', data.role);
localStorage.setItem('name', data.name);

// 5. Redirect to appropriate dashboard
const redirectUrl = data.role === 'teacher'
    ? './teacher/dashboard.html'
    : './student/dashboard.html';
window.location.href = redirectUrl;
}
...

```

### ### Backend: Password Hashing (Security)

```

```python
import hashlib

def hash_password(password: str) -> str:
    """Convert plain text password to irreversible hash"""
    return hashlib.sha256(password.encode()).hexdigest()

# Example:
# Input: "SecurePass123!"
# Output: "a3f5d2e8c..." (always same for same input)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Check if plain password matches stored hash"""
    return hash_password(plain_password) == hashed_password

# User enters: "SecurePass123!"
# Hashes to: "a3f5d2e8c..."
# Compare with stored: "a3f5d2e8c..." ✓ Match!
...

```

### ### Backend: Database Query

```

```python
conn = psycopg2.connect(
    host="localhost",
    database="school_management_system",

```

```

    user="postgres",
    password="your_password", # ⚠ Change this!
    port="5432"
)

cur = conn.cursor(cursor_factory=RealDictCursor)

# Execute parameterized query (prevents SQL injection)
cur.execute(
    'SELECT user_id, email, password, role, name FROM "User" WHERE email = %s AND role = %s',
    (request.email, request.role) # Parameters bound safely
)

user = cur.fetchone() # Get one result as dictionary
...

```

---

## ## 5. Error Handling & Edge Cases

### ### Case 1: Email Not Found

...

Frontend → "juan@invalid.com"  
 Database → No row found  
 Backend → 401 Unauthorized  
 Response → "Invalid email or password"  
 Frontend → Alert user  
 ...

### ### Case 2: Wrong Password

...

Frontend → Correct email, wrong password  
 Database → User found  
 Backend → Password hash mismatch  
 Backend → 401 Unauthorized  
 Response → "Invalid email or password" (intentionally vague for security)  
 ...

### ### Case 3: Email Already Registered

...

Frontend → Signup with "existing@example.com"  
 Database → Email exists in User table  
 Backend → 400 Bad Request  
 Response → "Email already registered"  
 Frontend → Alert user to login instead  
 ...

### ### Case 4: Database Connection Failed

...

Backend → Cannot connect to PostgreSQL

Backend → 500 Internal Server Error  
Response → "Database connection failed"  
Frontend → Alert: "Backend is down"  
...

---

## ## 6. Security Considerations

### ### ✓ What We're Doing Right

- **Password Hashing**: Passwords stored as SHA-256 hashes, not plain text
- **Parameterized Queries**: Using `%s` placeholders prevents SQL injection
- **HTTPS Ready**: Code structure supports HTTPS in production
- **Role-based**: Login filters by role (student/teacher)

### ### ⚠ Future Improvements Needed

- **Better Hashing**: Use bcrypt instead of SHA-256 (bcrypt is slower, harder to crack)
- **JWT Tokens**: Issue tokens so user stays logged in across pages
- **HTTPS**: Use HTTPS in production (not HTTP)
- **Rate Limiting**: Prevent brute force attacks on login
- **Email Verification**: Confirm email address is real before signup
- **Password Reset**: Allow users to recover forgotten passwords

---

## ## 7. How to Run

### ### 1. Update PostgreSQL Password

Edit [`backend/main.py`](`backend/main.py`) line 24:

```
```python
password="your_password", # ← Change to your actual password
```
```

### ### 2. Set Up Database

```
```bash
psql -U postgres -f static/learnsync.sql
```
```

This creates the `school_management_system` database and all tables.

### ### 3. Start Backend

```
```bash
cd backend
pip install fastapi uvicorn psycpg2-binary
python -m uvicorn main:app --reload
```
```

Backend runs at: `http://localhost:8000`

### ### 4. Open Frontend

Open [`static/login.html`](`static/login.html`) in browser (or serve via backend)

### 5. Test Login

...  
  
Role: student  
Email: test@example.com  
Password: TestPass123!  
...  
  
---

## 8. Local Storage (Frontend State)

After successful login, user data is stored in browser:

```
```javascript
// Access anywhere in your app
const userId = localStorage.getItem('user_id');      // "1"
const role = localStorage.getItem('role');            // "student"
const name = localStorage.getItem('name');            // "Juan Dela Cruz"

// To clear (on logout):
localStorage.clear();
```
```

Use this to:

- Display user name in dashboard header
  - Check role to show appropriate content
  - Persist login state across page refreshes
  - Validate that user is logged in before showing protected pages
- 

## 9. API Endpoints Summary

| Method | Endpoint         | Input                                                              | Output                                  | Purpose              |
|--------|------------------|--------------------------------------------------------------------|-----------------------------------------|----------------------|
| POST   | /api/auth/login  | {role, email, password}                                            | {success, message, user_id, role, name} | User login           |
| POST   | /api/auth/signup | {role, email, firstName, middleName, lastName, idNumber, password} | {success, message, user_id, role, name} | New account creation |
| GET    | /api/health      | None                                                               | {status}                                | Check backend status |